UNIVERSIDAD TECNOLÓGICA
FACULTAD DE INGENIERÍA EN SISTEMAS
CAMPUS VICTOR LEVIS SASSO
DESARROLLO DE SOFTWARE VII

Fecha Límite de Entrega: **I SEMESTRE 2026**
Del 15 al 17 de junio de 2026

Instructor: Ing. Irina Fong **Grupo: IGS131/ 1GS132 / 1GS133**

Guía de Laboratorio Práctico: "**CRUD - API Fetch y MySQL – (Guardar, Editar, Buscar)"**

https://www.youtube.com/watch?v=AXZGTOd8ASk

Convertir una matriz asociativa PHP a JSON	Las matrices asociativas son matrices que utilizan **claves con nombre** como índices para sus valores. $book = **array**("id"=>101, "label"=>"Lorem Ipsum"); **echo json_encode**($book);

Ejemplo:

```
        //$this->controlError[$field]
        $mensaje = "Producto no creado";
        $response = [ "success" => false,
                      "message" => $mensaje,
                      "accion" => "Guardar"];

        }

header("Content-Type: application/json"); // Asegúrate de enviar JSON
echo json_encode($response);
exit;
break;
```

A. Título de la experiencia

CRUD con Fetch + PHP OOP + MySQL
(Guardar, Editar y Buscar productos a través de un formulario dinámico)

API Fetch	**API Fetch** ofrece una forma sencilla y moderna de realizar **solicitudes HTTP** en **JavaScript** mediante **promesas** . Reemplaza métodos antiguos como **XMLHttpRequest** y permite una gestión eficiente de la recuperación de datos, **el análisis de** respuestas y la gestión de errores. Tanto si se usa **`.then()`** como **`async/await`,** Fetch facilita y agiliza el trabajo con **las API . Gracias a las comprobaciones de estado y la gestión de errores** adecuadas , ayuda a crear **aplicaciones web fiables e intuitivas.**

B. Temas

- Formulario HTML y envío asíncrono con **fetch**
- FormData y formato **multipart/form-data**
- Programación orientada a objetos en PHP
- Conexión segura a MySQL usando PDO
- Estructura de control **switch** en PHP y JavaScript
- Validación de entradas y manejo de errores
- Alertas amigables con **SweetAlert 2**

C. Objetivos

Tipo Descripción

"Implementar una aplicación web dinámica bajo la arquitectura cliente-servidor, integrando peticiones asíncronas con Fetch API y un backend robusto en PHP OOP con MySQL, enfocándose en la resolución de problemas técnicos y el aseguramiento de la integridad de los datos.

Objetivos Específicos:

- **Analizar y Refactorizar:** Evaluar el código base proporcionado para identificar posibles puntos de fallo y refactorizar la lógica de comunicación entre el frontend y el backend.
- **Optimizar el Flujo de Datos:** Diseñar una interfaz de comunicación mediante JSON que garantice respuestas limpias, eliminando redundancias y "ruido" en la salida del servidor.

- **Blindar el Código:** Aplicar buenas prácticas de seguridad (PDO para evitar inyecciones SQL) y validaciones preventivas tanto en el lado del cliente (JS) como en el servidor (PHP).
- **Depurar entornos asíncronos:** Utilizar herramientas del navegador (Inspector de Red/Consola) para rastrear fallos en el ciclo de vida de una petición HTTP y resolver errores de sintaxis o formato en tiempo real.
- **Modularizar la lógica:** Centralizar las operaciones CRUD en controladores mediante estructuras de control (switch) para mejorar la mantenibilidad y escalabilidad del proyecto.

D. Metodología

1. **Mini-explicación** (docente) sobre fetch, FormData y flujo petición-respuesta en PHP.
2. **Construcción guiada**:
    1. Maquetar formulario HTML.
    2. Programar **script.js** para capturar eventos y llamar a fetch.
    3. Reutilizar el **Modelo/conexion.php** (clase DB), crear el **Modelo/Productos.php** (clase Producto).
    4. Desarrollar **registrar.php** con switch ($accion) que reciba $_POST.
3. **Pruebas** de guardar, editar y buscar.
4. **Socialización y retroalimentación**.

E. Instrucciones paso a paso

1. **BD**: crear productosdb y la tabla

sql

CopiarEditar

```
CREATE TABLE productos (
  id INT AUTO_INCREMENT PRIMARY KEY,
  codigo   VARCHAR(20) NOT NULL,
  producto VARCHAR(100) NOT NULL,
  precio   DECIMAL(10,2) NOT NULL,
  cantidad INT NOT NULL
);  // si no agrega algún campo se le descontará 5 puntos.
```

2. **Archivos a crear según responsabilidades por el ejemplo presentado**

Carpeta/archivo	**Descripción breve**
Modelo/conexion.php	Clase DB (singleton o simple) que expone insertSeguro, updateSeguro, query.
/Productos.php	Clase Producto con propiedades y métodos guardar(), editar(), buscar(), validación.
registrar.php	Recibe $_POST, ejecuta switch($_POST['Accion']) → Guardar ó Modificar, retorna JSON success / message / accion / errors.

Carpeta/archivo	Descripción breve
index.html	Formulario con inputs: Código, Producto, Precio, Cantidad + botón *Registrar / Actualizar*.
script.js	Captura clic, crea FormData, envía con fetch, procesa respuesta con SweetAlert2.

3. **Validar** en PHP los campos obligatorios antes de guardar/editar.
4. **Mostrar** cada resultado con SweetAlert2 y recargar la tabla (función ListarProductos()).

Reglas del Negocio

Si estamos hablando de un catálogo o registro de productos comerciales, lo más común y con mayor sentido de negocio es que si vas a registrar un producto por primera vez, ingreses al menos 1 unidad existente.

🛒 ¿Por qué se suele usar min="1"?

Es el ideal para el formulario de Creación / Registro.

- Lógica: No tiene mucho sentido comercial registrar un producto nuevo en el sistema si no tienes stock disponible para venderlo. Forzar el min="1" asegura que todo producto nuevo nazca con existencia real en el inventario.



📦 ¿Por qué se suele usar min="0"? (El otro enfoque)

Es el estándar cuando el mismo formulario se reutiliza tanto para Guardar como para Modificar.

- Lógica: Imagina que registraste un producto con 10 unidades, pero se vendieron todas. El producto sigue existiendo en tu base de datos, pero su cantidad ahora es 0 (está agotado o *out of stock*).
- Si necesitas Modificar otro detalle de ese producto (como corregir el nombre o cambiar el precio) y tu formulario tiene un min="1", el navegador te dará un error de validación y no te dejará guardar los cambios del precio a menos que alteres la cantidad a 1 de forma obligatoria.

F. Recursos

- Video guía (referencial): https://youtu.be/AXZGTOd8ASk
- SweetAlert2 CDN:

html

CopiarEditar

<script src="https://cdn.jsdelivr.net/npm/sweetalert2@11"></script>

- Servidor local (WampServer / XAMPP).
- Editor de código (VS Code).

UNIVERSIDAD TECNOLÓGICA
FACULTAD DE INGENIERÍA EN SISTEMAS
CAMPUS VICTOR LEVIS SASSO
DESARROLLO DE SOFTWARE VII

Fecha Límite de Entrega: **I SEMESTRE 2026**
Del 15 al 19 de junio de 2026

Instructor: Ing. Irina Fong **Grupo: IGS131/ 1GS132 / 1GS133**

Estudiante: _____ **Fecha de Entrega: _____**

🧪 Guía de Laboratorio Práctico: "**CRUD - API Fetch y MySQL – (Guardar, Editar, Buscar)**"

G. Rúbrica de evaluación (100 pts)

Criterio	Puntos	PtsObtenidos
Formulario realiza guardar, editar y buscar correctamente	10	
Uso de Bootstrap para el diseño y botones (Agregar Alert)	10	
Lógica "Guardar / Modificar" centralizada con switch	15	
Uso de switch también en JavaScript	10	
Clases DB implementadas	10	
Clases Producto implementadas	10	
Validaciones + control de errores (servidor y cliente)	10	
Respuestas JSON con success, message, errors	10	
Uso de SweetAlert 2 para éxito y errores	10	
Código limpio, comentarios y estructura de carpetas	5	
Total	**100**	

Notas finales

- Puede ampliarse para incluir DELETE o paginación como reto extra.

- Puntualidad estaría dentro de la Primera, equivale a 15 puntos. Se le descontará si no es puntual.

Anexos

¿Cómo utilizar la API de obtención de JavaScript para obtener datos?

La **API Fetch** es una forma moderna de realizar **solicitudes HTTP** en **JavaScript.** Está integrada en la mayoría de los navegadores y permite a los desarrolladores realizar solicitudes de red (como obtener datos de un servidor) de forma sencilla y eficiente. La API Fetch reemplaza técnicas antiguas como **XMLHttpRequest** y **los métodos AJAX de jQuery** .

Sintaxis

```
fetch(url)
  .then(respuesta => respuesta.json())
  .then(datos => console.log(datos))
  .catch(error => console.error(error)
```

En la sintaxis anterior:

- **url** : la URL desde la que desea obtener datos.
- **.then()** : maneja la respuesta cuando la solicitud es exitosa.
- **.catch()** : captura cualquier error si la solicitud falla.

Cómo funciona Fetch

La función fetch() es una forma moderna de realizar solicitudes HTTP en JavaScript mediante promesas. Funciona así:

- **fetch(url)** envía una solicitud a la URL dada (el valor predeterminado es GET).
- Devuelve una **Promesa** que se resuelve en un objeto Respuesta.
- Utilice **.then()** para procesar la respuesta (por ejemplo, response.json()).
- Utilice **.catch()** para manejar errores de red.
- No arroja errores para códigos de estado **HTTP** como 404: verifique response.ok manualmente.

La función **fetch()** envía una solicitud a la API y recupera los datos del producto 1 de la URL proporcionada.

La respuesta se analiza en JSON con .then(response => response.json()) y los datos resultantes se registran en la consola, mientras que cualquier error se detecta y se muestra con **.catch().**

```
async function fetchData() {
    try {
        const response = await fetch('https://fakestoreapi.com/products/1');
        if (!response.ok) {
            throw new Error('Network response was not ok');
        }
        const data = await response.json();
        console.log(data);
    } catch (error) {
        console.error('Error:', error);
    }
}

fetchData();
```

- La función **fetch()** envía una solicitud a la API y recupera los datos del producto 1 de la URL proporcionada.
- La respuesta se analiza en JSON con .then(response => response.json()) y los datos resultantes se registran en la consola, mientras que cualquier error se detecta y se muestra con **.catch().**

Manejo del estado de respuesta HTTP

Gestionar el estado de respuesta HTTP en la API Fetch te ayuda a gestionar diferentes resultados según la respuesta del servidor, como códigos de éxito o de error. Puedes consultar el estado para determinar qué acción tomar.

```
fetch('https://fakestoreapi.com/products/1')
  .then(response => {
    if (!response.ok) {
      throw new Error('Network response was not ok');
    }
    return response.json();
  })
  .then(data => console.log(data))
  .catch(error => console.error('Error:', error));
```

Uso de async/await con Fetch API

Usar async/await con la API Fetch permite gestionar código asíncrono de forma más legible. Aquí tienes un ejemplo sencillo para obtener datos.

```
async function fetchData() {
  try {
    const response = await fetch('https://fakestoreapi.com/products/1');
    if (!response.ok) {
      throw new Error('Network response was not ok');
    }
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.error('Error:', error);
  }
}
```

Uso de async/await con Fetch API

- **Función asíncrona:** La función `fetchData` está marcada como `async`, lo que permite usar `await` dentro de ella. Esto hace que el código asíncrono se parezca más al código síncrono, mejorando la legibilidad.
- **await fetch():** La palabra clave `**await**` pausa la ejecución de la función hasta que se complete la solicitud `fetch()` y se reciba la respuesta. Esto evita la necesidad de varios métodos `.then()` y facilita la lectura del código.
- **await response.json():** Una vez recibida la respuesta, se usa `await` para analizar el cuerpo de la respuesta en formato JSON. Esto también garantiza que el código espere a que se complete el análisis de JSON antes de continuar.
- **Manejo de errores:** el uso de `try/catch` garantiza que cualquier error durante la solicitud de búsqueda o al analizar la respuesta se detecte y se gestione con elegancia, lo que evita que la aplicación se bloquee.

La gestión de errores en la API Fetch garantiza la correcta gestión de problemas como fallos de red o respuestas no válidas. A continuación, se muestra un ejemplo sencillo para demostrar cómo gestionar errores con Fetch.

```javascript
async function fetchData() {
    try {
        const response = await fetch('https://fakestoreapi.com/products/100');

        // Check if the response was successful
        if (!response.ok) {
            throw new Error('Network response was not ok');
        }

        // Parse the response body to JSON
        const data = await response.json();

        // Log the data
        console.log(data);
    } catch (error) {
        // Handle any errors that occurred during the fetch
        console.error('Error:', error);
    }
}

// Call the async function
fetchData();
```

💡 Consejos

Siempre que uses fetch(...).then(res => res.json()), asegúrate de que:

- Tu archivo PHP tenga **solo echo json_encode(...) como salida**.
- No haya echo, print, var_dump, ni espacios extra **fuera de eso**.
- El header("Content-Type: application/json"); esté al inicio del archivo.